

Examen Módulo 1

03/04/2023

Nombre: _____

Instrucciones

El presente examen tiene cuatro partes. Cada parte indica su puntaje y el nombre del archivo que se debe entregar. La tercera parte es opcional y otorga un punto de bono (nota máxima 8,0) a quienes hayan completado las primeras dos partes. Tendrás un tiempo de 110 minutos para resolver el examen. Debes entregar a través de Canvas un archivo para cada parte con el nombre que se especifica en el enunciado.

Parte 1 (Scratch; 2 puntos)

1. [.75] Crea un programa en Scratch (archivo **p1-1.sb3**) que pida al usuario un valor entero n y calcule y despliegue (el gatito debe decir) el resultado de la serie alternante de los primeros n impares:

$$1 - 3 + 5 - 7 + 9 - \dots$$

Puedes suponer que en el programa siempre se ingresará $n \geq 1$.

2. [.5] Gottfried Leibniz y otros matemáticos antes que él, determinaron que la constante π es aproximable mediante la siguiente serie:

$$1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \dots = \frac{\pi}{4}$$

Crea un programa en Scratch (archivo **p1-2.sb3**) que calcule y despliegue π considerando la serie anterior con n términos. Para esto, el programa debe pedir el valor n al usuario.

3. [.75] Crea una copia del programa anterior (archivo **p1-3.sb3**) que pida un valor n para aproximación de π usando la serie anterior, valores reales a y b , extremos del intervalo $[a, b]$ con (se puede suponer que siempre se ingresará $b > a$), y un valor real positivo w , tal que $w < 1$ (se puede suponer que siempre se ingresa válido). Tu programa debe generar una lista llamada **salida** con todos los valores de $\sin(a + kw\pi)$, en donde k es entero positivo que toma valores desde 0 hasta el mayor entero posible tal que $a + kw\pi \leq b$.

Nota importante: La función $\sin(x)$ en Scratch está en el último bloque de la sección “Operators” (buscar bloque que dice “abs”), que cuenta con múltiples opciones. Debes considerar que $\sin(x)$ en Scratch trabaja con ángulos en grados. Para usar ángulos en radianes como es el caso en este problema, debes multiplicar el ángulo por 180 y dividir por π para obtener el resultado correcto.

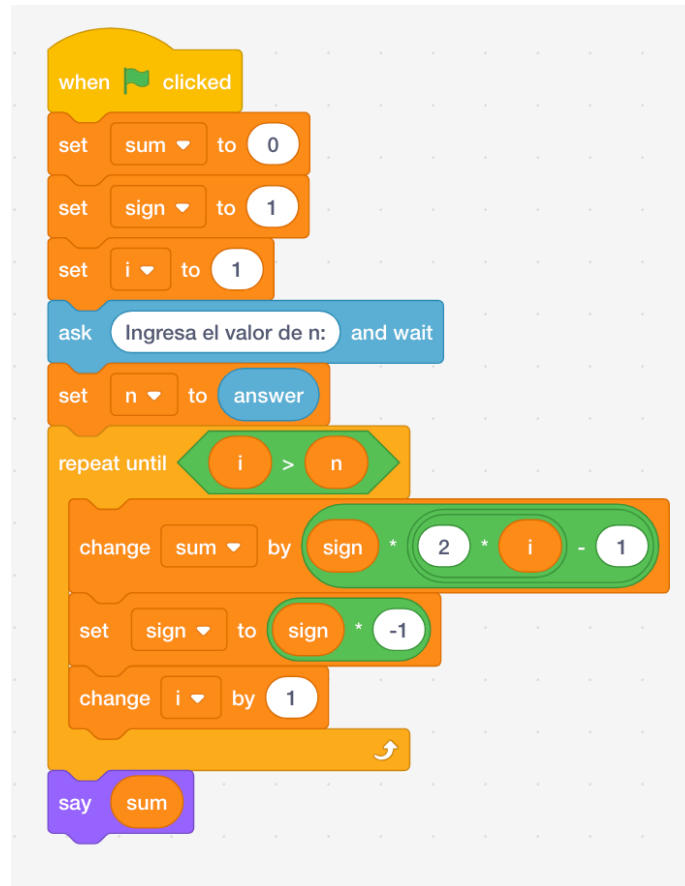
Solución

Figura 1: Solución de ítem 1

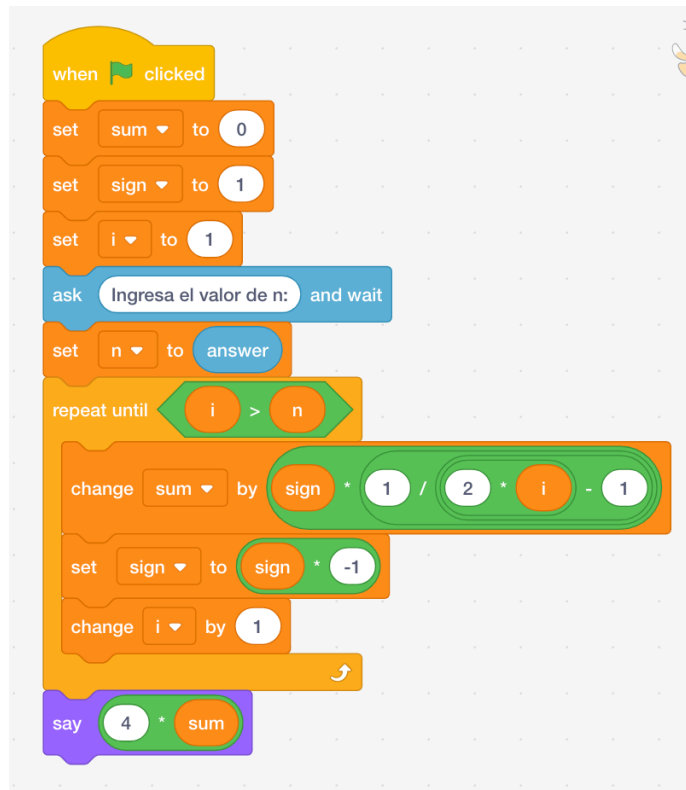


Figura 2: Solución de ítem 2

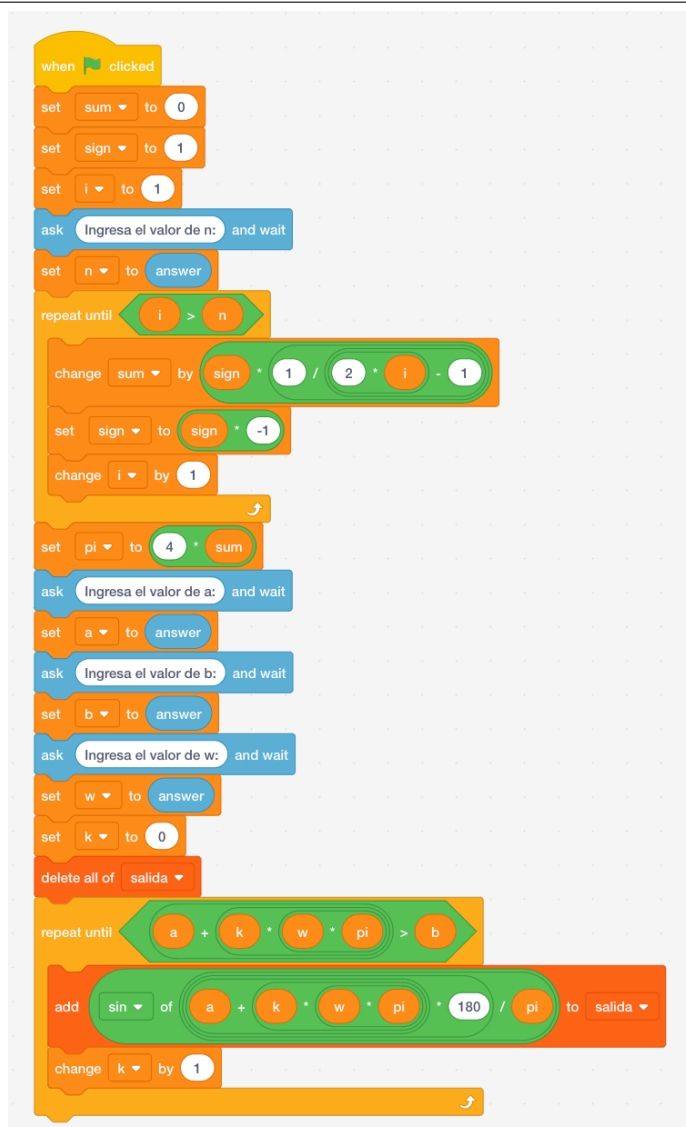


Figura 3: Solución de ítem 3

Puntaje

Se evalúa la completitud y correctitud de la solución en cada ítem del problema con un puntaje de apreciación en escala 0-2 (0: sin solución, o con errores graves, 1: solución incompleta o con errores leves, 2: excelente). Dicho puntaje es convertido a un ponderador entre 0 y 1: $0pt \rightarrow 0,0$, $1pt \rightarrow 0,5$, $2pts \rightarrow 1,0$). El ponderador obtenido es multiplicado por el puntaje máximo del ítem respectivo, con lo que se obtiene puntaje parcial del ítem. El puntaje total corresponde a la suma de los puntajes parciales.

Parte 2 (Scratch; p2.sb3; 2 puntos)

En esta segunda parte, simularás el movimiento circular uniforme de dos partículas A y B centradas en el origen, y con trayectorias de igual radio r . Las partículas tienen velocidades angulares distintas ω_A , ω_B , respectivamente, en rad/s.

Para lo anterior, considera que las posiciones de las partículas el plano cartesiano x - y son dadas por las

siguientes ecuaciones:

$$x_A(t) = r \cos(\omega_A t)$$

$$y_A(t) = r \sin(\omega_A t)$$

De las ecuaciones se entiende que en el tiempo t las coordenadas (x_A, y_A) de la partícula A están dadas por las funciones de coseno y seno, y la velocidad angular respectiva.

La posición inicial de ambas partículas es la misma, en $(r, 0)$.

En el caso de la partícula B , análogamente:

$$x_B(t) = r \cos(\omega_B t)$$

$$y_B(t) = r \sin(\omega_B t)$$

Igual que en el problema anterior, para usar la velocidad angular en rad/s con las funciones trigonométricas en Scratch, debes multiplicar la velocidad angular por 180 y dividir por π , que en este caso puedes manejar constante en 3.1416.

Requisitos:

- Crea en tu programa una lista llamada **parametros**, en donde deberás ingresar manualmente las constantes requeridas por las ecuaciones de movimiento, en el siguiente orden: r , ω_A y ω_B . Para probar tu programa, puedes ingresar manualmente a la lista los valores 4 (para r), 1.57 (para velocidad angular $\omega_A = \frac{\pi}{2}$), y 0.785 ($\omega_B = \frac{\pi}{4}$), considerando $\pi = 3.1416$.
- El programa al iniciarse debe pedir al usuario el tiempo total t a simular ($t \geq 1$, t entero), en segundos. Si el usuario ingresa un valor menor que 1, se le debe mostrar error y volver a pedir.
- Además, se debe pedir al usuario un valor entero f ($f \geq 1$, entero)¹. También se debe volver a pedir el valor f al usuario si se ingresa incorrectamente.
- La simulación tiene que desarrollarse iterativamente (con un ciclo), y en cada iteración de la simulación el tiempo simulado debe avanzar en $1/f$. La simulación debe iterar hasta que el tiempo transcurrido supere a t .
- En cada iteración, la simulación debe generar las posiciones de A y B en el tiempo actual simulado, usando las ecuaciones de movimiento mostradas arriba.
- El programa debe generar cuatro listas llamadas XA , XB , YA , YB , que contendrán todos los puntos cartesianos generados en las trayectorias de A y B en la simulación, con las ecuaciones de movimiento.
- Cada punto de la partícula A en la simulación debe quedar en las listas XA e YA . Para la partícula B , se deben usar las listas XB e YB . Al término de la simulación, todas las listas deben tener el mismo tamaño, considerando que se genera igual cantidad de puntos para cada partícula en la simulación.
- Encontrarás una demostración del funcionamiento del programa (YouTube) en <https://bit.ly/em1v1-202310-p2>

Nota: Es importante considerar que la simulación debe realizarse en forma instantánea, esto es, sin simular el tiempo real transcurrido. Así, en la práctica tomará mínimo tiempo al computador simular, por ejemplo, 60 segundos.

¹Esto corresponde a la frecuencia con que se generarán los puntos de las trayectorias de A y B . El valor de f está en Hz.

Desglose de puntaje:

- [.5] El programa pide los parámetros t y f al usuario, y valida cada uno de ellos. Es decir muestra un error cuando éstos son mal ingresados, y pide al usuario volverlos a ingresar, repitiendo este proceso hasta que ambos parámetros estén correctos.
- [.5] El programa implementa una iteración para simular el transcurso del tiempo en la simulación y la generación de puntos de trayectoria de partículas A y B con frecuencia f .
- [.5] El programa evalúa correctamente las ecuaciones de movimiento utilizando constantes provenientes de la lista `parametros`, y el tiempo simulado.
- [.5] El programa inicializa y genera correctamente los valores en las listas XA , YA , XB , YB .

Puntaje

Cada ítem es evaluado con una escala de 1-5 (1: no presenta solución, 2: presenta esbozo de solución incompleto o con errores graves, 3: solución incompleta o imprecisa, 4: error(es) mínimo(s), 5: excelente) y luego un ponderador (1:0, 2:0,25, 3:0,5, 4:0,75, 5:1,0) es aplicado al puntaje del ítem. El puntaje otorgado a la solución se calcula como la suma de los puntajes parciales obtenidos, aplicándose los ponderadores.

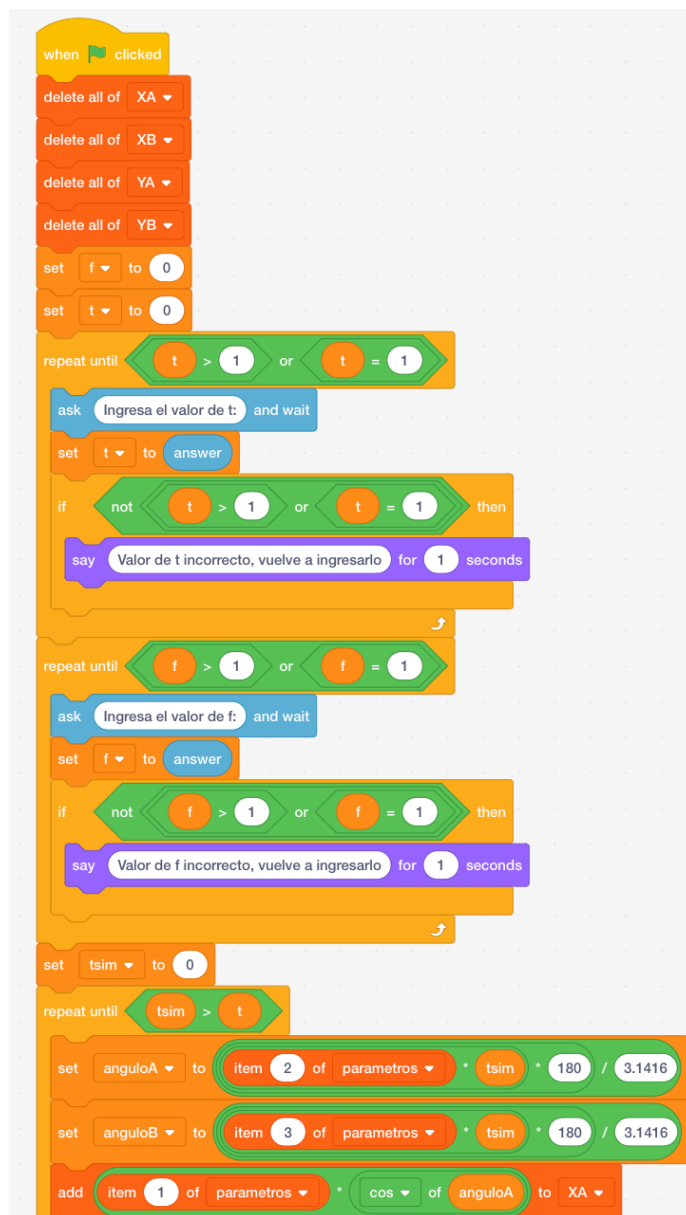
Solución

Figura 4: Solución



Figura 5: Continuación de la solución

Parte 3 (Scratch; p3.sb3; BONUS; 1 punto)

Esta parte del examen es opcional y sólo será evaluada si has logrado completar la Parte 2. Para continuar, crea una copia del programa de la parte anterior, con nombre **p3.sb3**.

Extiende el programa anterior para que determine y despliegue al finalizar la ejecución:

- [.5] La mayor distancia entre las partículas A y B y el instante de tiempo en que ésta ocurre. Usa para esto la distancia euclídeana d entre dos puntos (x_0, y_0) , (x_1, y_1) , definida como:

$$d = \sqrt{(x_1 - x_0)^2 + (y_1 - y_0)^2}$$

Para esto, sólo debes considerar los puntos de trayectoria generados en la simulación, y que se encuentran en las listas XA , XB , YA , e YB .

- [.5] La distancia euclídeana promedio entre las partículas A y B durante la simulación.

Ejemplo de ejecución:

Puedes encontrar un vídeo con una demostración aquí: <https://bit.ly/em1v1-202310-p3>

Puntaje

Se evalúa la completitud y correctitud de la solución en cada ítem del problema con un puntaje de apreciación en escala 0-2 (0: sin solución, o con errores graves, 1: solución incompleta o con errores leves, 2: excelente). Dicho puntaje es convertido a un ponderador entre 0 y 1: $0pt \rightarrow 0,0$, $1pt \rightarrow 0,5$, $2pts \rightarrow 1,0$). El ponderador obtenido es multiplicado por el puntaje máximo del ítem respectivo, con lo que se obtiene puntaje parcial del ítem. El puntaje total corresponde a la suma de los puntajes parciales.

Solución

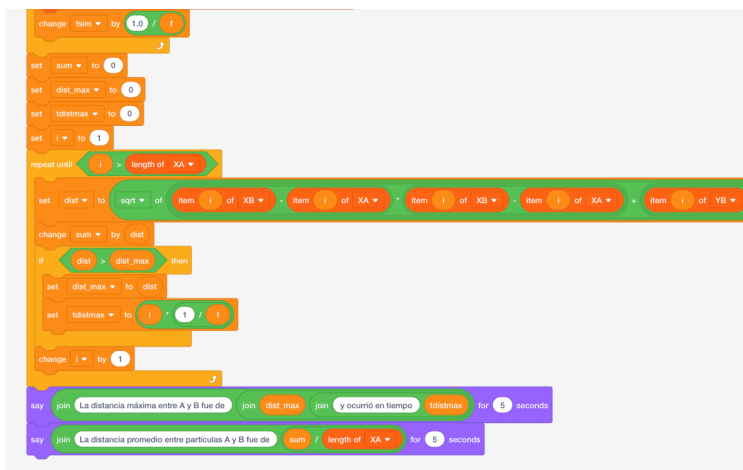


Figura 6: Solución de la Parte 3

Parte 4 (Python; p1.py; 2 puntos)

Escribe un programa en Python que haga lo siguiente:

- [.5] Pida al usuario las coordenadas de tres puntos en el plano cartesiano y las convierta a formato decimal (x_0, y_0) , (x_1, y_1) , (x_2, y_2) .
- [.5] Calcule las longitudes de los lados del triángulo formado por los puntos. Para esto, considerar la distancia euclidiana según la fórmula dada en la parte 3 (ver arriba).
- [1.0] Muestre si el triángulo es isósceles (dos lados iguales) o escaleno (todos los lados de largo diferente), desplegando en líneas separadas el resultado de evaluar cada criterio. Ejemplo:

```
Ingrese el valor de x0: 0
Ingrese el valor de y0: 0
Ingrese el valor de x1: 2
Ingrese el valor de y1: 4
Ingrese el valor de x2: 4
Ingrese el valor de y2: 0
Isosceles? True
Escaleno? False
```

Puntaje

Se evalúa la completitud y correctitud de la solución en cada ítem del problema con un puntaje de apreciación en escala 0-2 (0: sin solución, o con errores graves, 1: solución incompleta o con errores leves, 2: excelente). Dicho puntaje es convertido a un ponderador entre 0 y 1: $0pt \rightarrow 0,0$, $1pt \rightarrow 0,5$, $2pts \rightarrow 1,0$). El ponderador obtenido es multiplicado por el puntaje máximo del ítem respectivo, con lo que se obtiene puntaje parcial del ítem. El puntaje total corresponde a la suma de los puntajes parciales.

Solución

Listing 1: Solución de la Parte 4

```
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3
4  x0 = 0
5  y0 = 0
6  x1 = 0
7  y1 = 0
8  x2 = 0
9  y2 = 0
10
11  msg = "Ingrese el valor de "
12  x0 = float(input(msg + "x0:"))
13  y0 = float(input(msg + "y0:"))
14  x1 = float(input(msg + "x1:"))
15  y1 = float(input(msg + "y1:"))
16  x2 = float(input(msg + "x2:"))
17  y2 = float(input(msg + "y2:"))
18
19  d01 = ((x1-x0)**2+(y1-y0)**2)**.5
20  d02 = ((x2-x0)**2+(y2-y0)**2)**.5
21  d12 = ((x2-x1)**2+(y2-y1)**2)**.5
22
23  print(d01, d02, d12)
24
25  print("Isosceles? ", (d01 == d02 and d01 != d12) or\
26        (d02 == d12 and d12 != d01) or\
27        (d01 == d12 and d12 != d02))
28  print("Escaleno? ", (d01 != d02) and (d02 != d12) and (d01 != d12))
```